

Web Systems Development



Agenda

- Homework 1 – Outline and Questions
- Playwrite

Homework 1 Tips

- Update SQL file as suggested
 - Prompting for syntax: Give table structure, indicate MySQL as database.
- Build MySQL Docker Container (see examples 1 and 2)
 - Same structure as example
- Build Apache Docker
 - Slight difference. Can ask Gordon for assistance.
- When building a Docker container, you have to refer to the docker file. In the examples, we were in the same directory. (.)
- Fix the Docker compose file
 - See example 5
 - Google “LAMP Docker Compose”
 - Ask Gordon to help wire together

.env files

The values live in .env. MySQL reads them by one set of names; your PHP reads them by another. `docker-compose.yml` is where you connect the two.

.env -- copied, password changed

```
MYSQL_ROOT_PASSWORD=root
MYSQL_DATABASE=app
MYSQL_USER=appuser
MYSQL_PASSWORD=change_me
APP_PORT=8080
ADMINER_PORT=8081

gitignored. Never in a commit.
```



docker-compose.yml -- you write

```
db:
  environment:
    MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
    MYSQL_DATABASE: ${MYSQL_DATABASE}
    MYSQL_USER: ${MYSQL_USER}
    MYSQL_PASSWORD: ${MYSQL_PASSWORD}
web:
  environment:
    DB_HOST: db
    DB_NAME: ${MYSQL_DATABASE}
    DB_USER: ${MYSQL_USER}
    DB_PASSWORD: ${MYSQL_PASSWORD}
```



config.php -- given

```
$dbHost = getenv("DB_HOST")
?: "db";

$dbName = getenv("DB_NAME")
?: "app";

$dbUser = getenv("DB_USER")
?: "appuser";

$dbPass = getenv("DB_PASSWORD")
?: "";

?: is the fallback. "" is not
your password: wire it up.
```

- The MySQL image creates the database and the user from the `MYSQL_*` variables on first start. Your PHP never sees those names — it asks for `DB_USER` and `DB_PASSWORD`. Same value, new name, and `${...}` in compose is the only place the two meet.
- Symptom when the web half is missing: the page loads, the form submits, and PHP says access denied for 'appuser'@'...' using password: NO. That NO is the empty-string fallback.

Readings — Testing

- **Chojrin, Modern Testing with PHP — chapters 1, 2, 7**
[\(RPI library\)](#)
 - 1 — what a test is for; 2 — your first PHPUnit test; 7 — the Grocy case study, which is Homework 2
- **Playwright — Writing tests** **playwright.dev**
 - A robot that uses your site. Read it; we run it in class.
- **Ian Cooper — TDD, Where Did It All Go Wrong** **[\(video, 1 hr\)](#)**
 - Most people test the wrong unit. This is the talk I will argue with today.
- **Vocke — The Practical Test Pyramid (three sections)**
 - Why many fast tests and few slow ones is not a matter of taste

ITWS 2110 · FROM PANTRY TO GROCY

Writing Playwright tests

Every pattern you need for Homework 2 is already in Tuesday's Pantry tests. This deck reads them line by line, then shows the Grocy version of each.

Reading: playwright.dev/docs/writing-tests · Code: `inclass/wk3/pantry/tests/{api,e2e}/`

A test is a function that gets a fixture

test(name, async ({ request }) => { ... }) — the name is a sentence; request is an HTTP client; the body is arrange, act, assert.

TUESDAY'S FIRST API TEST

```
test('POST /api/products creates a product that GET returns',
  async ({ request }) => {
    const created = await createProduct(request,           // arrange
      { best_before: '2026-12-01' });

    const fetched = await request.get(`/api/products/${created.id}`); // act

    expect(fetched.status()).toBe(200);                      // assert
    expect(await fetched.json()).toMatchObject({
      id: created.id, name: created.name, amount: 0, best_before: '2026-12-01',
    });
  });
```

THE PIECES

test	one test, registered
'POST ...'	the sentence in the report
{ request }	a FIXTURE: an HTTP client
await	every call is async
expect	the assertion library

No browser anywhere in this file. Same three beats as StockTest; the thing under test is the HTTP contract.

Every test starts from nothing — so make your own data

Each test gets a fresh browser context. Your data is the one thing it does NOT reset — so every test creates its own product.

PANTRY -- the helper every API test calls

```
async function createProduct(request, extra = {}) {
  const name = `Apples ${Date.now()}-${random}`;
  const res = await request.post('/api/products',
    { data: { name, ...extra } });
  expect(res.status()).toBe(201);
  return res.json();
}
```

GROCY -- same helper, two more fields

```
async function newProduct(request, name) {
  const [loc] = await (await
    request.get('/api/objects/locations')).json();
  const [unit] = await (await
    request.get('/api/objects/quantity_units')).json();
  const r = await request.post('/api/objects/products', { data: {
    name, location_id: loc.id, qu_id_purchase: unit.id, qu_id_stock:
    unit.id,
  } });
  return (await r.json()).created_object_id;
}
```

Pantry wiped its database before every run. Grocy keeps yours — so the unique name matters more, not less. workers: 1 in both: one SQLite file.

Two kinds, and the await is the difference

api -- expect(value): checks once, now

```
expect(res.status()).toBe(422);
expect(fetched.status()).toBe(200);
expect((await after.json()).amount).toBe(3.5);
expect((await res.json()).error).toMatch(/not enough/i);
```

a response is already a value; nothing to wait for

e2e -- await expect(locator): keeps checking, up to 5 s

```
await expect(row).toBeVisible();
await expect(row.locator('.amount')).toHaveText('9');
await expect(row.locator('.badge')).toHaveText('fresh');
await expect(page.locator('.err')).toHaveText('Not enough in
stock');
```

the page changes after a click; this waits for it

- On Tuesday you renamed `.amount` to `.qty` and all three browser tests went red after five seconds each. That five seconds was `toHaveText` retrying. The message named the locator and printed Expected / Received — the same message you will read on Grocy.
- Grocy's refusal is a 400, not a 422; the shape of the test is identical: status, then message, then confirm the stock did not move.

The refusal test you already wrote, and its Grocy twin

PANTRY -- went green when you fixed Stock::consume()

```
test('consuming more than is in stock is refused with 422',
  async ({ request }) => {
    const { id } = await createProduct(request);          // arrange
    await request.post(`/api/products/${id}/purchase`,
      { data: { amount: 1 } });

    const res = await request.post(                        // act
      `/api/products/${id}/consume`, { data: { amount: 2 } });

    expect(res.status()).toBe(422);                       // assert
    expect((await res.json()).error).toMatch(/not enough/i);
    const p = await (await request.get(
      `/api/products/${id}`)).json();
    expect(p.amount).toBe(1);                             // shelf
    untouched
  });
```

GROCY -- HW2 Task 3, test 3. You fill in the blanks.

```
test('consuming more than is in stock is refused',
  async ({ request }) => {
    const id = await newProduct(request, `Rice ${Date.now()}`);
    await request.post(`/api/stock/products/${id}/add`,
      { data: { amount: 1 } });

    const res = await request.post(
      `/api/stock/products/${id}/consume`, { data: { amount: 2 } });

    expect(res.status()).toBe(____);                      // found in Task 0
    expect((await res.json()).error_message).toMatch(/___/);
    const stock = await (await request.get(
      `/api/stock/products/${id}`)).json();
    expect(stock.stock_amount).toBe(____);
  });
```

Same rule, same shape, different app. Swagger at /api tells you the paths and the field names.

Same file shape, different fixture: a page

Swap { request } for { page } and the test becomes a person: open, type, click, look. Arrange, act, assert, unchanged.

TUESDAY'S FIRST BROWSER TEST

```
test('a user can add a product and see it in the table', async ({ page }) =>
{
  const name = `Milk ${Date.now()}`; // arrange

  await page.goto('/'); // act
  await page.fill('#new-product input[name="name"]', name);
  await page.click('#new-product button[type="submit"]');

  const row = page.locator('#products tr', { hasText: name }); // assert
  await expect(row.locator('.amount')).toHaveText('0');
  await expect(row.locator('.badge')).toHaveText('fresh');
});
```

THE PIECES

test	one test, registered
'a user...'	the sentence in the report
{ page }	a FIXTURE: a fresh tab
{ request }	what you used all morning
await	every call is async
expect	the assertion library

Same three beats as StockTest. The act is what a person does; the assert is what a person sees.

Point at a thing, then do to it what a person would

BUYING AND USING CHANGE THE AMOUNT ON THE SHELF

```
const row = page.locator('#products tr', { hasText: name }); // the row
with Eggs in it

const buy = row.locator('form[action$="/purchase"]'); // the Buy
form IN THAT ROW
await buy.locator('input[name="amount"]').fill('12');
await buy.locator('button').click();
await expect(row.locator('.amount')).toHaveText('12');

const use = row.locator('form[action$="/consume"]');
await use.locator('input[name="amount"]').fill('3');
await use.locator('button').click();
await expect(row.locator('.amount')).toHaveText('9');
```

THE SAME MOVES ON GROCY

```
await page.goto('/product/new');
await page.fill('#name', name);
await page.selectOption(
  '#location_id', { index: 1 });
await page.click('#save-product-button');

page.locator('#stock-overview-table tr',
  { hasText: name })
```

- A locator is a description, not a lookup. Nothing runs until an action or an expect uses it — then Playwright waits for the element to exist, be visible, be enabled. Auto-waiting; no sleep() anywhere on Tuesday or in HW2.
- Scope it. row.locator('.amount') is one cell; page.locator('.amount') is seven. Find the row by its text, then look inside it. Grocy's stock table works the same way.
- Grocy adds selectOption: three dropdowns that start blank, so { index: 1 } picks the first real choice.

Arrange over the API, assert in the browser

Pantry's browser tests typed every product into the form. Grocy's form has three dropdowns. Ask for both fixtures and let the API do the setup.

HW2 tests/e2e -- the purchase story

```
test('a user can purchase a product and see it in stock', async ({ page, request }) => {
  const name = `E2E Milk ${Date.now()}`;
  const id = await newProduct(request, name); // arrange: API, ~20 ms
  await request.post(`/api/stock/products/${id}/add`, { data: { amount: 3 } }); // act: the purchase

  await page.goto('/stockoverview'); // assert: what a person sees
  const row = page.locator('#stock-overview-table tr', { hasText: name });
  await expect(row).toContainText('3');
});
```

- Compare Pantry's test 2: it clicked Buy in the browser because the click was cheap and the story was about the page. Here the click costs three dropdowns and the story is still about the page — so buy over the API and look in the browser.
- Put the click in the browser only when the click is the thing under test.

Each test still makes its own product. The API just does it faster than the form would.

You have already read this failure once

WHAT YOU SAW WHEN `.amount` BECAME `.qty`

```
X 1 tests/e2e/pantry.spec.js:7:1 > a user can add a product
... (5.2s)
```

```
Error: expect(locator).toHaveText(expected) failed
Locator: locator('#products tr').filter({ hasText: 'Milk...' })
        .locator('.amount')
Expected: "0"
Timeout: 5000ms
Error: element(s) not found

Error Context: test-results/.../error-context.md
```

HOW TO READ IT, ON GROCY TOO

```
5.2 s   it waited the whole timeout. Not flaky: gone.
element(s) not found   the selector matches nothing here.
Received ""            it matched, but the text is empty.
```

```
test-results/<test>/test-failed-1.png   the page at that moment
test-results/<test>/error-context.md     the page as an ally
tree
```

```
docker compose run --rm e2e npx playwright test --trace on
then drop trace.zip on trace.playwright.dev
```

- Wrong selector, wrong page, or the app really is broken — the screenshot tells you which in two seconds. On Grocy, wrong selector is the usual one; open the page's HTML and look for the id.

Everything maps

	Pantry (Tuesday)	Grocy (HW2)
Create a product	<code>POST /api/products {name}</code>	<code>POST /api/objects/products {name, location_id, qu_id_purchase, qu_id_stock}</code>
Buy / use	<code>POST /api/products/{id}/purchase · /consume {amount}</code>	<code>POST /api/stock/products/{id}/add · /consume {amount}</code>
Refusal	<code>422, {error: 'Not enough in stock'}</code>	<code>400, {error_message: '...cannot be > current stock...'}</code>
Read stock	<code>GET /api/products/{id} → amount</code>	<code>GET /api/stock/products/{id} → stock_amount</code>
The form	<code>#new-product, one field</code>	<code>/product/new, #name + three selectOption</code>
The table	<code>#products tr, hasText</code>	<code>#stock-overview-table tr, hasText</code>
Where the ids come from	<code>you knew them</code>	<code>ask: GET /api/objects/locations, /quantity_units – or read Swagger at /api</code>

Reminder

- HM1 Due Today